Guia do código: como o áudio atravessa o programa

Este guia segue a ordem em que o programa realmente corre. A ideia é conseguires apontar para uma função e explicar o que acontece a seguir, sem teres de apresentar a implementação como uma caixa negra.

Visão geral

```
Ficheiro WAV no computador
        |
        |  host/pico_host.py envia PCM por USB
        v
main.c: usb_read_exact()
        |
        v
main.c: denoiser_process_hop()
        |
        v
 denoiser.c: janela -> FFT -> estimativa de ruído
              -> cálculo de ganhos -> IFFT -> reconstrução
        |
        v
main.c: usb_write_exact()
        |
        |  USB devolve PCM ao computador
        v
host/pico_host.py guarda filtrado.wav
```

O USB é usado como um cabo de transporte de dados. O computador continua a ser responsável por ler e reproduzir ficheiros WAV. A Pico recebe amostras, processa-as e devolve amostras; não precisa de saber o que é um ficheiro WAV.

---

1. `main.c`: o programa começa em `main()`

1.1 Inicializar USB e esperar pelo computador

A primeira instrução importante é:

```
stdio_init_all();
```

O Pico SDK inicializa os canais de entrada/saída configurados no CMake. Neste projeto ativámos USB CDC, por isso a entrada e a saída standard (`getchar`, `printf`, etc.) passam pela porta série USB.

Depois:

```
while (!stdio_usb_connected()) tight_loop_contents();
```

A Pico espera até o computador abrir a porta série USB. Só depois escreve mensagens. Isto evita perder a mensagem inicial antes de o terminal se ligar.

1.2 Criar o estado do denoiser

A variável global:

```
static Denoiser denoiser;
```

reserva memória para o estado do algoritmo. É global porque a estrutura contém os buffers da FFT, o histórico de amostras, a estimativa de ruído e a sobreposição entre blocos. Não queremos colocar uma estrutura deste tamanho na stack pequena da função `main()`.

A chamada:

```
denoiser_init(&denoiser);
```

prepara esse estado. Mais à frente explicamos o que essa função inicializa.

1.3 A Pico anuncia que está pronta

O programa imprime uma linha `READY` antes de ficar à espera de comandos através de `read_line()`.

Não é seguro o script Python depender apenas dessa mensagem: ela pode ter sido enviada antes de o computador abrir a porta série. Por isso, o script começa por enviar `P\n`. A Pico responde `PONG` e, no ciclo seguinte, volta a imprimir `READY`. Assim, o computador confirma que a Pico está a ouvir, mesmo que a primeira mensagem `READY` já tenha passado.

Existem três comandos:

- `P`: confirmar que a ligação USB está ativa; a Pico responde `PONG`;
- `B`: correr um benchmark com um sinal gerado dentro da Pico;
- `S N`: receber `N` blocos e aplicar redução de ruído;
- `U N`: receber `N` blocos e fazer FFT/IFFT sem aplicar ganhos (teste de reconstrução);
- `E N`: devolver os mesmos bytes recebidos, sem DSP (teste de USB).

A vantagem é termos dois modos no mesmo firmware, sem misturar mensagens de texto com bytes de áudio durante o processamento.

---

2. Modo `B`: benchmark sintético

Quando a Pico recebe `B`, `main()` chama `run_benchmark()`.

2.1 Gerar o sinal de teste

Para cada bloco, `make_test_sample()` cria uma amostra:

- nos primeiros 62 blocos, gera apenas ruído;
- nos blocos seguintes, soma ruído com uma sinusóide de 440 Hz.

A geração do sinal fica fora da medição de tempo. Assim, não estamos a medir o custo de `sinf()` nem do gerador aleatório; estamos a medir o algoritmo de redução de ruído.

2.2 Medir a função principal

A linha:

```
uint32_t start = time_us_32();
```

guarda o instante inicial. Logo a seguir é chamada:

```
denoiser_process_hop(&denoiser, input_hop, output_hop);
```

No fim, outra chamada a `time_us_32()` permite calcular o tempo gasto nessa função.

No relatório final:

- `average` é a média dos tempos;
- `best` é o mais rápido;
- `worst` é o mais lento;
- `over budget` conta os blocos acima de 16 000 µs.

A 16 kHz, 256 amostras representam 16 ms. Por isso, 16 000 µs é o orçamento de cálculo para cada chamada, se quisermos processar continuamente um bloco novo a cada 16 ms.

O benchmark não mede qualidade de áudio e não mede transferência USB.

---

3. Modo `S N`: receber e devolver áudio pelo USB

Este é o modo que permite testar gravações reais sem ligar um microfone à Pico.

3.1 O computador prepara o áudio

Em `host/pico_host.py`, a função `process_wav()` abre o WAV e verifica quatro coisas:

1. áudio PCM sem compressão;
2. um canal (mono);
3. 16 bits por amostra;
4. 16 000 amostras por segundo.

Se faltar alguma condição, o script pede conversão. Depois calcula quantos blocos de 256 amostras são necessários e acrescenta zeros ao último bloco, se necessário.

Cada amostra ocupa 2 bytes. Portanto:

`256 amostras × 2 bytes = 512 bytes por bloco`

O script envia primeiro o comando `S N\n`, onde `N` é o número de blocos.

3.2 A Pico responde `START`

Em `main.c`, o comando é interpretado com `sscanf()`. O primeiro carácter escolhe o modo: `S` (redução normal), `U` (ganho unitário) ou `E` (eco USB). Depois é chamada `run_usb_stream(N, mode)`.

Nos modos `S` e `U`, a função chama `denoiser_init()`. Isto é importante: cada gravação começa com o histórico do algoritmo limpo, em vez de herdar o ruído estimado da gravação anterior. No modo `E`, não é necessário inicializar o denoiser.

Depois a Pico imprime `START` e começa o ciclo de blocos.

3.3 `usb_read_exact()`: receber um bloco

A função recebe exatamente `sizeof(input_hop)` bytes. Como `input_hop` tem 256 elementos `int16_t`, isso corresponde a 512 bytes.

A função não assume que o USB entrega os 512 bytes de uma só vez. Vai acumulando os bytes recebidos até completar o bloco.

Depois desses bytes serem colocados no array `input_hop`, a Pico tem 256 amostras de áudio mono, assinadas, de 16 bits.

3.4 `denoiser_process_hop()`: aplicar a redução de ruído

Agora entramos em `denoiser.c`. Esta é a função central do projeto. Recebe 256 amostras novas e devolve 256 amostras processadas. O modo `U` chama `denoiser_process_hop_unity()`, que usa o mesmo percurso de FFT/IFFT e overlap-add, mas não calcula nem aplica ganhos de redução de ruído.

A função faz as etapas seguintes, nesta ordem.

A. Atualizar o bloco de 512 amostras

A FFT trabalha com 512 amostras, mas só recebemos 256 novas de cada vez.

O `memmove()` desloca as 256 amostras mais recentes da chamada anterior para o início de `input_history`. O `memcpy()` coloca as 256 amostras novas na segunda metade.

Assim, cada janela tem 512 amostras e partilha metade com a janela anterior. A 16 kHz:

- janela de 512 amostras = 32 ms;
- avanço de 256 amostras = 16 ms;
- sobreposição = 50%.

B. Aplicar a janela

`arm_mult_q15()` multiplica cada amostra pela janela sqrt-Hann guardada em `window_q15`.

Isto reduz as descontinuidades nas extremidades do bloco antes de calcular a FFT. A mesma janela é usada na reconstrução.

C. Preparar a entrada da FFT

A CMSIS-DSP espera os números complexos organizados de forma intercalada:

`parte real, parte imaginária, parte real, parte imaginária, ...`

Como o áudio original é real, a parte imaginária começa a zero. O ciclo copia as 512 amostras para `spectrum` nesse formato.

D. Passar do tempo para as frequências

```
arm_cfft_q15(&d->fft, d->spectrum, 0u, 1u);
```

Esta é a FFT direta de 512 pontos. Em vez de vermos a evolução da amplitude no tempo, passamos a ter informação sobre a quantidade de sinal em cada frequência.

Usamos 257 posições distintas entre DC e Nyquist. As restantes posições fazem parte da simetria do espectro de um sinal real.

E. Medir a potência de cada frequência

`calculate_power()` calcula, em cada bin, a magnitude ao quadrado:

`potência = parte_real² + parte_imaginária²`

Depois suaviza a potência no tempo: 60% do valor anterior e 40% do valor novo. Guarda também o valor mínimo observado no bloco de estimação atual.

A potência é usada em vez da amplitude porque a fórmula da subtração espectral foi escrita em termos de potência.

F. Estimar o ruído e calcular os ganhos

`calculate_gains()` compara a potência atual de cada frequência com a estimativa do ruído nessa frequência.

A estimativa guarda mínimos em blocos de 16 frames e mantém até cinco blocos anteriores, além do bloco atual. Cada frame avança 16 ms; 16 frames correspondem a 256 ms. Cinco blocos completos correspondem a cerca de 1,28 s de histórico, mais o bloco em construção.

Para cada frequência, `calculate_bin_gain()` calcula o ganho:

`G = max(1 - sqrt(4 × potência_do_ruído / potência_do_sinal), 0,05)`

A interpretação é simples:

- se o ruído estimado for pequeno face ao sinal, o ganho fica próximo de 1 e a frequência é pouco alterada;
- se o ruído estimado for elevado, o ganho diminui;
- o ganho nunca desce abaixo de 0,05, para evitar apagar completamente uma frequência.

Depois há duas suavizações:

- suavização em frequência: média entre o bin anterior, o atual e o seguinte;
- suavização no tempo: 40% do ganho anterior e 60% do ganho calculado agora.

Estas suavizações reduzem alterações abruptas nos ganhos.

G. Aplicar os ganhos ao espectro

`apply_symmetric_gains()` atribui o mesmo ganho às frequências simétricas do espectro. Isto mantém a simetria necessária para reconstruir um sinal real.

A função CMSIS-DSP `arm_cmplx_mult_real_q15()` multiplica cada valor complexo pelo seu ganho real.

H. Voltar ao domínio do tempo

```
arm_cfft_q15(&d->fft, d->spectrum, 1u, 1u);
```

Esta é a FFT inversa. O resultado volta a ser um bloco de amostras no tempo.

A implementação Q15 da transformada tem uma escala interna. Por isso, o código multiplica o resultado por 512 (2^9) e limita os valores ao intervalo permitido por `int16_t`.

I. Aplicar a janela de síntese e juntar os blocos

`arm_mult_q15()` aplica novamente a janela sqrt-Hann. Depois, o ciclo de overlap-add soma o bloco reconstruído ao conteúdo que ficou guardado em `overlap`.

O programa copia as primeiras 256 amostras acumuladas para `output` e desloca a segunda metade do acumulador para preparar a chamada seguinte.

É assim que os blocos com 50% de sobreposição voltam a formar áudio contínuo.

J. Atualizar o histórico do estimador de ruído

No fim da função, incrementam-se os contadores de frames. Quando chegam a 16 frames, `finish_noise_block()` guarda os mínimos desse bloco no histórico e começa a recolher mínimos para o bloco seguinte.

A função termina com 256 amostras processadas no array `output_hop`. Como a janela usa 50% de sobreposição e o histórico começa a zero, o primeiro bloco

de saída é maioritariamente silêncio; é a latência inicial do processamento por blocos.

3.5 `usb_write_exact()`: devolver o áudio ao computador

`usb_write_exact()` envia os 512 bytes de `output_hop` através da USB CDC. Usa a variante raw de `stdio_put_string()`, sem conversão de caracteres ou de mudanças de linha.

É importante que isto seja binário: uma amostra de áudio pode conter qualquer byte, incluindo zero, `\n` e `\r`. Não pode ser tratada como texto.

A Pico repete o processo até receber todos os blocos anunciados no comando `S N`.

No fim envia uma linha `DONE` com o número de blocos e os tempos do DSP. O script do computador recebe essa linha e grava o WAV de saída.

---

4. O que acontece no script do computador

O script `host/pico_host.py` tem duas tarefas possíveis.

Benchmark

Ao abrir a porta série, envia `P\n`, espera por `PONG` e depois por `READY`. Com `--benchmark`, envia `B\n` e imprime as linhas que a Pico devolve até aparecer `BENCHMARK_DONE`.

Processar um WAV

Com `--input entrada.wav`, o script:

1. valida o formato WAV;
2. calcula o número de blocos;
3. envia `S N\n`;
4. espera por `START`;
5. envia um bloco de 512 bytes;
6. lê os 512 bytes correspondentes à saída;
7. repete até acabar o ficheiro;
8. espera por `DONE`;
9. guarda as amostras recebidas num novo WAV.

O script envia um bloco e espera pela resposta antes de enviar o seguinte. É uma escolha deliberadamente simples para evitar sobrecarregar os buffers USB.

---

5. Parâmetros que podes alterar sem reescrever tudo

No topo de `denoiser.h`:

- `DENOISER_SAMPLE_RATE_HZ`: frequência de amostragem, atualmente 16 kHz;
- `DENOISER_FRAME_SIZE`: tamanho da janela, atualmente 512;
- `DENOISER_HOP_SIZE`: novas amostras por chamada, atualmente 256;
- `DENOISER_NOISE_BLOCKS`: número de blocos antigos mantidos na estimativa de ruído, atualmente 5;
- `DENOISER_FRAMES_PER_BLOCK`: frames usados em cada bloco de mínimos, atualmente 16.

Atenção: alterar o tamanho da FFT pode exigir outra função de inicialização CMSIS-DSP e outra correção de escala. Não basta mudar o número 512.

Os parâmetros da subtração espectral encontram-se no início de `denoiser.c`: sobressubtração (`OVERSUB_NUM`), ganho mínimo, suavização da potência e suavização do ganho.

---

6. O que esta prova de conceito demonstra — e o que não demonstra

Demonstra que é possível transportar amostras PCM entre o computador e a Pico, processar cada bloco com o algoritmo escolhido e recuperar um WAV para ouvir e analisar.

Não demonstra ainda a aquisição contínua de um microfone, a saída para auscultadores/colunas, nem a cadência de um sistema de áudio USB em tempo real. Também não prova que o ruído de qualquer gravação será removido sem artefactos.

---

8. Os três testes de áudio e o que cada um isola

Usa sempre o mesmo WAV de entrada e executa estes testes pela ordem indicada.

Teste 1: USB sem DSP (`E`)

Na Pico:

```
memcpy(output_hop, input_hop, sizeof(output_hop));
```

Este modo não inicializa nem chama o denoiser. A Pico limita-se a devolver os 512 bytes do bloco que recebeu. O script guarda `eco.wav`.

Resultado esperado: `eco.wav` deve ser igual ao WAV original, amostra por amostra. Se não for, o problema está na transferência USB ou no programa do computador, e ainda não faz sentido investigar a FFT.

Teste 2: FFT/IFFT sem redução (`U`)

A Pico chama `denoiser_process_hop_unity()`. Esta função faz: 1. atualização do histórico de amostras; 2. janela de análise; 3. FFT; 4. IFFT sem aplicar ganhos; 5. janela de síntese; 6. overlap-add.

Assim, conseguimos testar a reconstrução sem a estimativa de ruído nem a subtração espectral.

Resultado esperado: `unity.wav` deve soar praticamente igual ao original. Pode existir erro pequeno de quantização e o script compensa o atraso inicial de 256 amostras (16 ms). Se este teste estiver incorreto, o problema está na FFT/IFFT, na escala Q15 ou na reconstrução.

Teste 3: redução de ruído (`S`)

Este é o modo normal. Entre a FFT e a IFFT, o código estima a potência por frequência, calcula os ganhos e aplica-os ao espectro. Só depois reconstrói o sinal.

Resultado esperado: em gravações com ruído adequado ao modelo, o ruído deve diminuir sem degradar demasiado a fala. Não há garantia de melhoria para todas as gravações.

Comandos no computador

```
# USB apenas
python host/pico_host.py --port /dev/ttyACM0 --input input.wav --output echo.wav --echo

# Reconstrução FFT/IFFT sem redução
python host/pico_host.py --port /dev/ttyACM0 --input input.wav --output unity.wav --unity

# Algoritmo completo
python host/pico_host.py --port /dev/ttyACM0 --input input.wav --output denoised.wav
```

O script desloca os resultados dos modos `U` e `S` 256 amostras para compensar o primeiro bloco de saída usado como latência de arranque. O modo `E` não tem latência de DSP e não é deslocado.